Case study 2

Let us consider an example shown in program-3 below, which attempts to modify the 'string literals' and will lead to undefined behaviour.

```
 1 #include <stdio. h>
 2
 3 int main ()
 4 {
 5     /*Initialize the pointer to the base address of the
string*/
 6     char *p = "string constant";
 7
 8     /*Attempt to modify the character at 0th index*/
 9     *(p + 0) = 'S';
10
11     return 0;
12 }
```

In the code shown in program-3 above, one is trying to modify the string literal which is pointed to by the pointer 'p'. Since program-3 is non-compliant code when compiled and run, it leads to undefined behaviour, as the standard imposes absolutely no requirements and the compiler can do anything. The behaviour is undefined if a program attempts to modify any character in a string literal. Modifying a string literal frequently results in an access violation because the former is typically stored in read-only memory.

The output obtained when compiled in gcc is shown in Figure 5. Some compilers will issue the warnings while some will silently compile the code, as in case of the gcc compiler, where even the *-Wall* option is enabled.

Any attempt to modify a C string literal will lead to undefined behaviour.

Case study 3

Instance 1: According to the C standards, signed integer overflow is undefined behaviour. Some compilers may trap the overflow condition when compiled with some trap handling options while some simply ignore the overflow conditions, assuming that the overflow will never happen, and generate the code accordingly.

```
 1 #include <stdio.h>
 2
 3 int main()
 4 {
 5     /* Assigning the INT_MAX value to the 'x' variable */
 6     int x = INT_MAX;
 7
 8     /* Checking if wraps around or not */
 9     if (x + 1 > x)
10     {
11         x++;
12     }
13     else
14     {
15         /* Error handling code */
16     }
17
18     return 0;
19 }
```

In program-4 given above, we are checking whether the value of 'x', which is *INT_MAX* after adding Constant 1, results in overflow or not. Since the behaviour in this case is undefined by the C standards, the compiler may ignore the 'if' condition by optimising the code, which means it may simply replace the condition *(x + 1)* by the TRUE value, assuming that the value of *(x + 1)* is always greater than 'x', or may proceed with the code generation.

In the gcc compiler, it simply wraps around and gives the negative value when the value of *x = INT_MAX*. The assembly code generated by the gcc compiler for program-4 is shown in Figure 6.

```
satya@Elegance: ~/Codes
[Satya]~/Codes
gcc program_3.c
[Satya]~/Codes
./a.out
Segmentation fault (core dumped)
[Satya]~/Codes
```

Figure 5: Output of program-3

```
0000000004004ed <main>:
#include <stdio.h>
#include <limits.h>

int main()
{
  4004ed:       55                      push   %rbp
  4004ee:       48 89 e5                mov    %rsp,%rbp
        /* Assigning the INT_MAX value to the 'x' variable */
        int x = INT_MAX;
  4004f1:       c7 45 fc ff ff ff 7f    movl   $0x7fffffff,-0x4(%rbp)

        /* Checking if wraps around or not */
        if (x + 1 > x)
  4004f8:       8b 45 fc                mov    -0x4(%rbp),%eax
  4004fb:       83 c0 01                add    $0x1,%eax
  4004fe:       3b 45 fc                cmp    -0x4(%rbp),%eax
  400501:       7e 04                   jle    400507 <main+0x1a>
        {
                x++;
  400503:       83 45 fc 01             addl   $0x1,-0x4(%rbp)
        else
        {
                /* Error handling code */
        }

        return 0;
  400507:       b8 00 00 00 00          mov    $0x0,%eax
}
```

Figure 6: Assembly code generated for program-3, compiled using gcc compiler

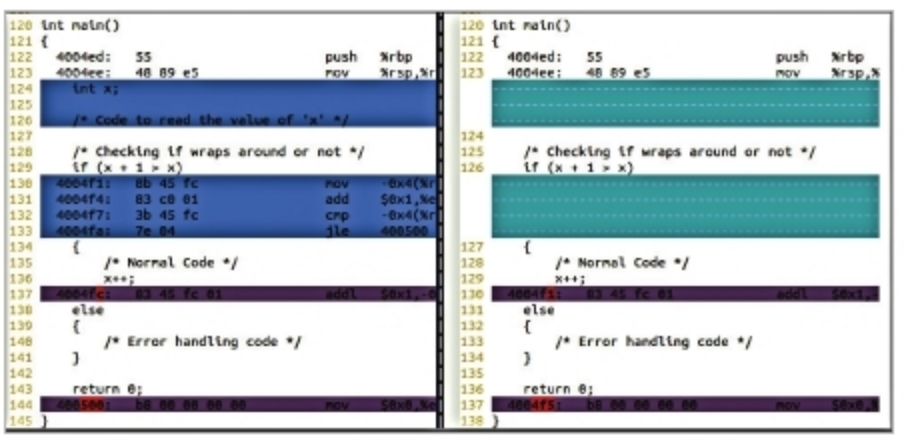

Figure 7: Assembly code generated for program-4, compiled using the gcc compiler with options *-fno-strict-overflow* and *-fstrict-overflow*